

Plateforme de données streaming pour **Casablanca** : matching rider/véhicule, prévision de la demande et observabilité à l'échelle de la ville. Méthodologies, réalisations et conclusions ; livrable accompagné du code source et d'extraits de données.

Architecture Kappa / 16 conteneurs Docker / Kafka / Flink / Spark / Cassandra / MinIO / FastAPI / Grafana

Code source : github.com/mohamedamineelabidi/CasaMotion

■ Contexte & objectifs

Problème. Casablanca (~4M d'habitants) ne dispose d'aucune couche de données partagée pour sa mobilité : grands/petits taxis et minibus circulent sans GPS, sans réservation ni horaire. Conséquences : chauffeurs à vide, attente des usagers, absence d'interopérabilité (rail/BRT/taxi), paiement en espèces sans historique exploitable. *Le véritable manque est une couche de données reliant l'offre à la demande, pas un manque de véhicules.*

Objectifs (cahier des charges).

- Matching **rider-véhicule** en moins de 5 s.
- Prévision de la **demande par zone** à 30 min.
- **Visibilité ville** : heatmaps, KPI, zones sous-desservies.
- **Planification** pilotée par la donnée.

Chaîne attendue : ingestion temps réel · traitement *event-time* · base de service · ML batch · dashboards · API sécurisée.

■ Architecture & choix techniques

Pipeline (Kappa). Producteurs → Kafka (KRaft) → 3 jobs Flink → Cassandra → Grafana / FastAPI. **Kafka** est la source de vérité unique et rejouable ; MinIO est le *data lake* S3 ; Spark est strictement **hors-ligne** (batch + ML). C'est une architecture **Kappa** (une seule couche de traitement), et non Lambda.

Choix	Justification
Kappa	1 couche ; Kafka rejouable
Kafka KRaft	sans ZooKeeper, bascule + rapide
Flink	fenêtres <i>event-time</i> , latence < s
Cassandra	écriture optimisée, TTL natif
MinIO	API S3 (Spark/Flink/Connect)
H3 res-9	lookup zone en O(1)

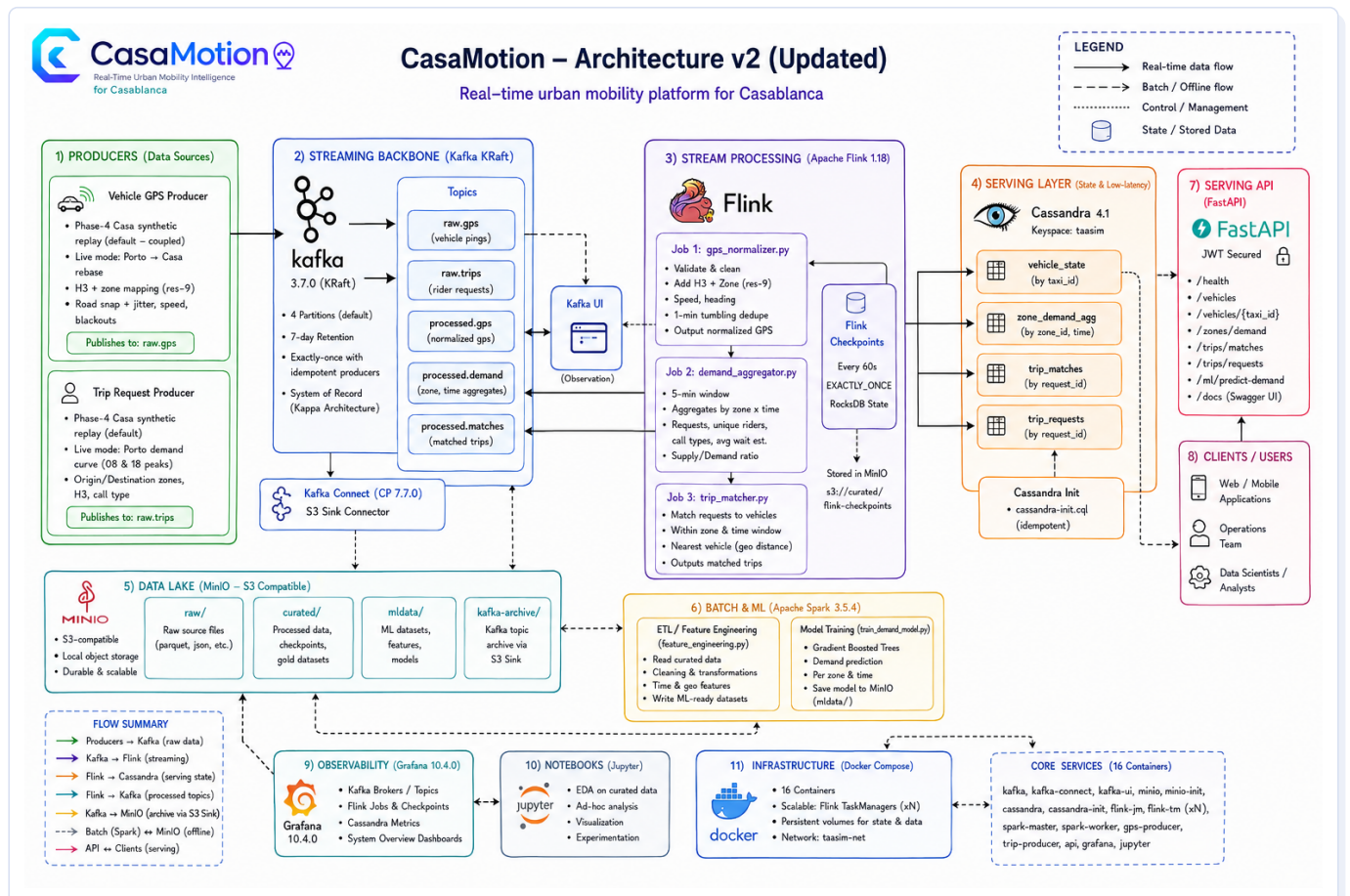


Figure 1 — Architecture de bout en bout : producteurs → Kafka (KRaft) → 3 jobs Flink → Cassandra ; data lake MinIO, batch Spark/ML hors-ligne, API FastAPI et observabilité Grafana.

■ Méthodologie — étapes du projet

Semaine 1 — Fondations & géographie

- Stack Docker (Kafka, MinIO, Cassandra, Flink, Spark, Grafana, Jupyter) ; datasets **Porto** (1,8 GiB) + **NYC TLC** chargés dans MinIO.
- Découpage en 16 zones : 9 polygones administratifs OSM + 7 cellules de Voronoï (banlieues nord), nettoyés des chevauchements et bords maritimes.
- Score d'activité A–E par zone : $\text{score}_z = 0,40 \rho + 0,30 c + 0,20 d + 0,10 t$ (densité de population HCP 2024, commerce et diversité POI via Glovo, hubs de transit OSM), normalisé puis découpé en quantiles.
- Lookup H3 res-9 pré-calculé (`h3_zone_lookup.json`) : affectation de zone en **O(1)** au lieu d'un *point-in-polygon*.
- Warping Porto → Casablanca : transformation affine bbox, ancrage au centroïde + jitter, puis routage **OSRM** sur le réseau routier réel de Casablanca.

Semaine 2 — Couche de stockage

- Kafka Connect (S3 Sink) : archivage `raw.gps` / `raw.trips` vers MinIO, partitionné par heure.
- Schéma Cassandra conçu autour des requêtes : 3 tables avec TTL ; partition `date_bucket` pour borner la taille. **ADR v1** documentant chaque décision.

Semaine 3 — Traitement temps réel (PyFlink)

- Job 1 — Normalizer GPS : validation, déduplication, zone H3, **anonymisation** (centroïde + jitter haché) → `vehicle_positions`.
- Job 2 — Agrégateur de demande : fenêtres glissantes 30 s par zone, ratio offre/demande → `demand_zones`.
- Job 3 — Matcher : *keyBy* zone, pool de véhicules en MapState, *fan-out* vers zones adjacentes + déduplication → `trips`.
- Garanties : checkpoints RocksDB **EXACTLY_ONCE** (60 s) vers MinIO ; *watermarks* (3 min / 10 s + idleness) pour les événements tardifs.

Semaine 5 — ETL batch (Spark)

- Porto : 1 710 670 → 1 660 794 trajets valides → Parquet curated (43 MiB).
- NYC TLC : 9,38 M → 8,81 M → **192 K lignes de demande**. Règle Kappa : NYC reste batch-only, ne transite jamais par Kafka et n'entraîne pas le modèle.

Semaine 6 — Machine Learning + API

- Feature engineering : matrice de **183 981 lignes** ; cible = trajets par zone par créneau de 30 min.
- Anti-fuite (leakage) : suppression de `supply` / `supply_demand_ratio` (issus du même créneau que la cible) ; conservation des seules variables rétrospectives (lags 1j / 7j, moyenne mobile 7j).
- Split temporel (et non aléatoire) au `2014-05-01` : entraînement 151 911 / test 32 070.
- Modèle : Spark MLlib `GBTRegressor` (maxDepth=5, maxIter=50), *pipeline* StringIndexer → VectorAssembler → GBT.

■ Données & artefacts (extraits fournis)

Artefact	Rôle	Volume / format
<code>data/train.csv</code>	Source Porto (polylines GPS, donneur)	1,71 M trajets · CSV
<code>data/nyc-tlc/*.parquet</code>	Motifs de demande NYC (batch-only)	9,38 M lignes · Parquet
<code>data/zone_mapping_v4.csv</code>	16 zones + score A–E + centroïdes	16 lignes · CSV
<code>data/h3_zone_lookup.json</code>	Index H3 res-9 → zone (O(1))	JSON
<code>curated/trips/</code>	Trajets nettoyés (sortie Spark ETL)	43 MiB · Parquet
<code>mldata/features/</code>	Matrice d'apprentissage (anti-leakage)	183 981 lignes · Parquet
<code>mldata/models/demand_v1/</code>	Pipeline GBT entraîné (StringIndexer+GBT)	Spark MLlib

Flux de bout en bout : `raw.gps` / `raw.trips` → Flink (3 jobs) → `vehicle_positions` · `demand_zones` · `trips` (Cassandra) ; en parallèle `Kafka Connect` archive vers MinIO et `Spark` produit `curated/` puis `mldata/` consommé par `FastAPI`.

■ Réalisations & résultats mesurés

Métrique	Cible	Mesuré
Matching (P95)	< 5 s	~1,2 s ✓
Fraîcheur GPS	< 15 s	~4 s ✓
MAJ demande	30 s	30 s glissant ✓
ETL Porto (1,7 M)	< 5 min	43 MiB Parquet ✓
Checkpoints Flink	60 s	25 conséc., 0 échec ✓
Charge API forecast	<500 ms	test à venir ◊

Modèle de prévision (GBT) vs baseline naïve (lag 7j) :

Métrique	GBT	Naïf
RMSE	3,71	6,84
MAE	2,11	—
R ²	0,75	—

Amélioration RMSE : -45,8%
Battu globalement et sur les 16 zones.

Validation pilotée par la preuve. *Healthcheck* démo (16 vérifications : conteneurs, jobs Flink RUNNING, fraîcheur des tables) ; tests de réalisme du mouvement (8/8 distincts, 139/139 en mouvement, 200/200 < 100 m, 80/80 dans la bbox) ; test d'événements tardifs / *watermark* ; validation ML (split temporel, table de comparaison à la baseline par zone).

■ Conclusions & perspectives

Conclusions.

- Plateforme **temps réel de bout en bout** et reproductible (`docker compose up -d` , 16 conteneurs) couvrant ingestion, traitement *event-time*, service, ML batch et observabilité.
- Cibles de performance **atteintes ou dépassées** ; modèle ML **supérieur à la baseline** sur toutes les zones.
- **Confidentialité by design** : les coordonnées brutes ne sont jamais persistées (centroïde + jitter).

Limites assumées (honnêteté méthodologique).

- Données **donneuses** : mouvement réel de Porto + motifs de demande NYC, ré-ancrés via HCP 2024 + Glovo (pas de jeu de données taxi Casablanca ouvert).
- 7 des 16 zones sont des **cellules de Voronoï** (échafaudage d'agrégation), non des arrondissements officiels.
- L'API sert par défaut une **heuristique** ; le modèle GBT Spark se charge en option (`PYSPARK_ENABLED=1`).

Perspectives (S7-S8). Durcissement sécurité (rotation JWT, TLS, SASL/ACL Kafka) ; tests d'intégration CI (P95 < 5 s) ; test de charge Locust ; démo de reprise sur checkpoint.



Figure 2 — Pipeline d'apprentissage (Semaine 6) : trajets curated → feature engineering anti-fuite (183 981 lignes) → split temporel (2014-05-01) → GBTRegressor → modèle servi (RMSE 3,71 ; -45,8% vs baseline).

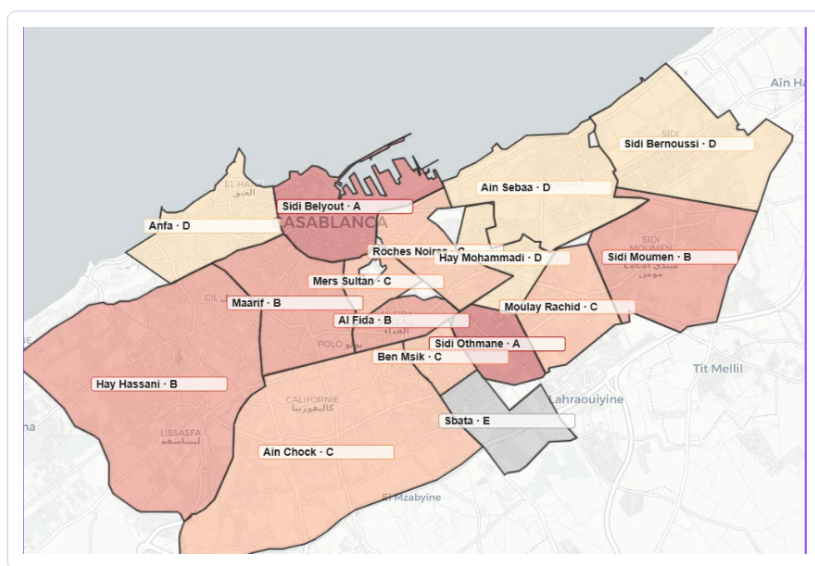


Figure 3 — Découpage de Casablanca en 16 zones (9 arrondissements OSM + 7 cellules de Voronoï) borné par la bbox de la ville.